

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

Construye | Proyecto MiBici: propuesta y base reproducible | Colaborativo

Equipo 03

Anaí Lael Barrera Magaña — A01797380

Gerardo González Martínez — A01840096

Miguel Ángel Ramírez Castillo — A01840045

Sebastián Estrada García — A00805402

Operaciones de aprendizaje automático

18 de octubre de 2026

Repositorio: https://github.com/mna-mlops-tc5061-03/TC5061-Proyecto-Equipo03

Integrante	Matrícula	Rol principal
Miguel Ángel Ramírez Castillo	A01840045	Ingesta reproducible y administración del repositorio
Gerardo González Martínez	A01840096	Configuración, registro común en MLflow, variables y experimento base
Anaí Lael Barrera Magaña	A01797380	Exploración de datos y README
Sebastián Estrada García	A00805402	ML Canvas, diagramas, referencias, evidencia de MLflow y documento de la propuesta

Introducción

MiBici, el sistema de bicicleta pública del Área Metropolitana de Guadalajara, publica cada mes los viajes de sus usuarios en su portal de datos abiertos (MiBici, s.f.-a). En esta primera entrega proponemos cómo responder con Machine Learning tres preguntas de su operación: cuántos viajes iniciarán mañana en cada estación (P1, regresión), qué estaciones estarán en riesgo de quedarse sin bicicletas o sin anclajes en las siguientes tres horas (P2, clasificación) y qué grupos de estaciones comparten patrones de uso (P3, clustering).

Además dejamos la base reproducible sobre la que seguirá el proyecto: un comando que descarga, limpia y versiona los datos; un módulo de variables que no usa información del futuro, y un baseline y un modelo candidato por pregunta, registrados en MLflow.

1. Formulación del problema

Llenamos el ML Canvas (Dorard, 2015) para acordar qué se predice, para quién y qué decisión se toma con el resultado antes de construir los modelos (Dorard, 2018). Después lo actualizamos (iteración 2) con las cifras reales de la ingesta. La Figura 1 muestra el Canvas y la Tabla 1 resume la formulación de cada pregunta.

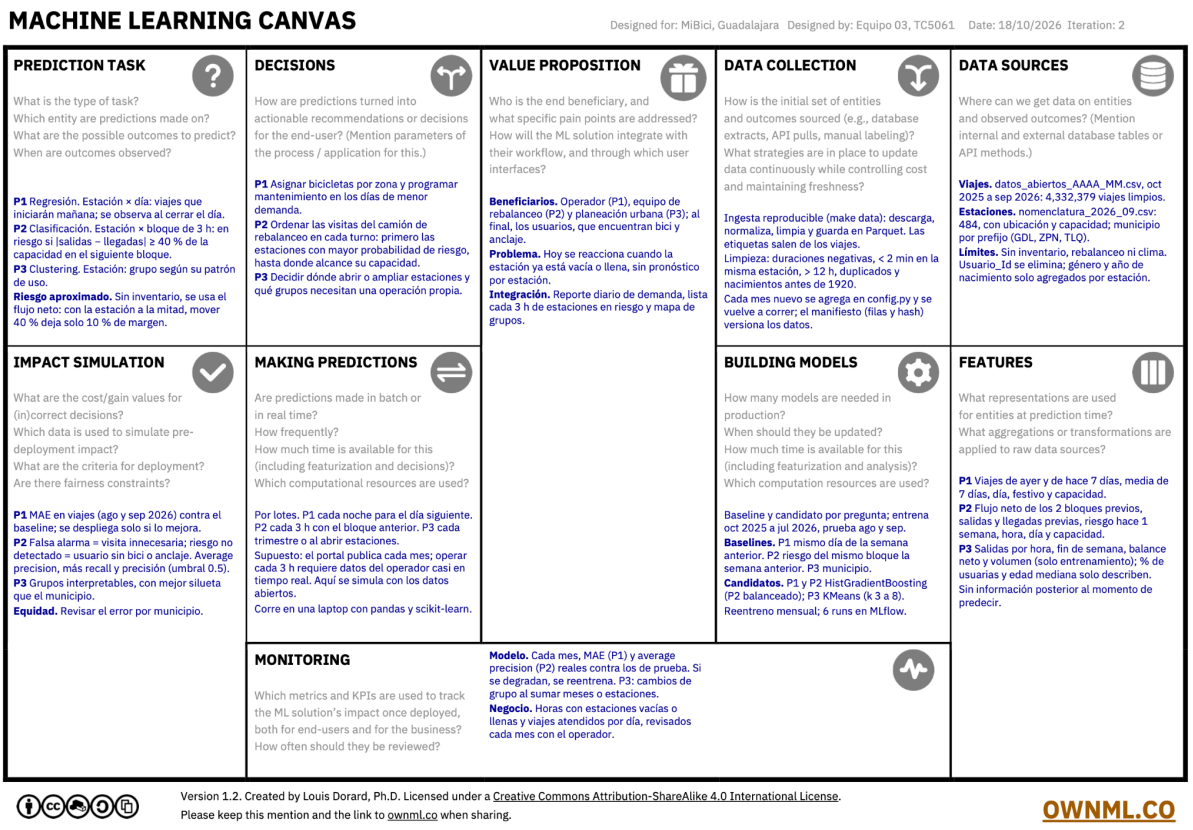


Figura 1. *ML Canvas del proyecto (iteración 2), sobre la plantilla de Dorard (2015).*

Característica	P1 · Demanda	P2 · Riesgo	P3 · Grupos
Tipo de tarea	Regresión	Clasificación binaria	Clustering
Qué se predice o agrupa	Viajes que iniciarán en la estación	Si la estación quedará en riesgo de vaciarse o llenarse	Grupo de la estación según su patrón de uso
Granularidad	Estación × día	Estación × bloque de 3 horas	Estación
Horizonte	Día siguiente	Siguiente bloque de 3 horas	Sin horizonte: se recalcula cada trimestre o al abrir estaciones
Cuándo se observa el resultado	Al cerrar el día	Al cerrar el bloque	No hay etiqueta real

Característica	P1 · Demanda	P2 · Riesgo	P3 · Grupos
Quién lo usa	Operador del sistema	Equipo de rebalanceo	Planeación urbana y expansión
Qué decisión toma	Cuántas bicicletas asigna a cada zona y qué días programa el mantenimiento (los de menor demanda)	En qué orden visita el camión las estaciones en cada turno: primero las de mayor probabilidad de riesgo, hasta donde alcance su capacidad	Dónde abrir o ampliar estaciones y qué grupos necesitan una operación propia

Tabla 1. *Formulación de las tres preguntas como problemas de ML.*

Por qué es relevante. Sin un pronóstico por estación, la operación solo puede reaccionar cuando una estación ya está vacía o llena. En P1, la demanda está concentrada: el 20 % de las estaciones genera el 55 % de los viajes (notebook ***1.0-ab-exploracion-datos***), así que repartir la flota según la demanda esperada de cada estación pesa más que repartirla de forma pareja. En P2, una estación vacía deja al usuario sin bicicleta y una llena lo obliga a buscar otro anclaje, para lo que el sistema solo le da 10 minutos adicionales sin cargo (MiBici, s.f.-b); anticipar el riesgo un bloque antes da tiempo de mandar el camión. En P3, los grupos muestran qué estaciones se usan de forma parecida aunque estén en municipios distintos, lo que orienta la expansión y la operación por zona.

1.1 Cómo aproximamos el riesgo de P2

MiBici no publica cuántas bicicletas hay en cada estación, así que no podemos observar si una estación se vació. Lo aproximamos con el flujo neto de cada bloque de tres horas, llegadas menos salidas: si salen muchas más bicicletas de las que llegan, la estación se vacía, y si llegan muchas más, se llena. Una estación está en riesgo en un bloque cuando:

$$|\text{llegadas} - \text{salidas}| \geq 0.4 \times \text{capacidad de la estación}$$

- **Por qué 40 %**. Como no conocemos la ocupación al inicio del bloque, suponemos una estación a la mitad: un movimiento neto del 40 % de su capacidad la deja con solo 10 % de margen antes de quedar vacía o llena.
- **Por qué bloques de 3 horas**. Es un horizonte con el que el equipo de rebalanceo puede planear las visitas de un turno, reúne suficientes viajes para que el flujo no sea solo ruido y divide el día en ocho bloques.
- **Limitaciones**. La etiqueta no ve los movimientos del camión de rebalanceo ni la ocupación real: una estación que empieza llena y pierde el 40 % no queda vacía. Además, juntamos vaciarse y llenarse en una sola clase; separarlas queda como

mejora. Excluimos las estaciones con capacidad 0, porque en ellas no se puede calcular la fracción.

El umbral y el tamaño del bloque viven en **config.py** (**UMBRAL_RIESGO**, **BLOQUE_HORAS**) y se registran como parámetros en cada run de P2, así que se pueden ajustar con el operador. Con esta definición, el 4.8 % de los bloques de entrenamiento y el 4.4 % de los de prueba quedan en riesgo.

2. Datos

2.1 Rango de fechas

Usamos los 12 meses completos más recientes, del 1 de octubre de 2025 al 30 de septiembre de 2026, por tres razones:

- **Un ciclo anual completo.** El periodo incluye todas las temporadas, las vacaciones y los días de descanso obligatorio, que cambian la demanda.
- **La red actual.** La nomenclatura que publica MiBici es la de septiembre de 2026; con años anteriores habría viajes en estaciones que ya cambiaron, se movieron o no existían.
- **Millones de registros que caben en una computadora personal.** Los 390 MB de CSV quedan en 31 MB de Parquet particionado por año y mes, unas 12 veces menos, y en memoria ocupan unos 400 MB. Los modelos no entrenan sobre los viajes individuales, sino sobre tablas agregadas: 140,402 filas en P1 (estación × día), 1,122,997 en P2 (estación × bloque de 3 horas) y 426 en P3 (una por estación). En la laptop de uno de los integrantes, con 7 GB de RAM, el pico de memoria al entrenar fue de unos 1.9 GB por pregunta y cada una corrió en menos de 30 segundos. Ampliar el periodo a 2022–2026 multiplicaría el volumen por cinco, unos 20 millones de viajes, y llevaría el pico cerca de 8 GB (estimación). Para ese caso, el Parquet particionado permite leer solo las columnas o los meses necesarios (The pandas development team, s.f.).

Las fechas están en **config.py** (**FECHA_INICIO**, **FECHA_FIN**). Para agregar un mes basta con sumar su URL y volver a correr **make data**.

2.2 Fuente

Los datos vienen del portal de datos abiertos de MiBici (MiBici, s.f.-a): https://mibici.net/es/datos-abiertos/. Cada archivo tiene un identificador propio en su URL, así que fijamos los enlaces en **config.py** en lugar de leer la página, que puede cambiar.

Archivo	Enlace
Viajes octubre 2025	https://mibici.net/site/assets/files/115104/datos_abiertos_2025_10.csv
Viajes noviembre 2025	https://mibici.net/site/assets/files/116773/datos_abiertos_2025_11.csv
Viajes diciembre 2025	https://mibici.net/site/assets/files/118153/datos_abiertos_2025_12.csv
Viajes enero 2026	https://mibici.net/site/assets/files/120163/datos_abiertos_2026_01.csv
Viajes febrero 2026	https://mibici.net/site/assets/files/121761/datos_abiertos_2026_02.csv
Viajes marzo 2026	https://mibici.net/site/assets/files/123236/datos_abiertos_2026_03.csv
Viajes abril 2026	https://mibici.net/site/assets/files/124927/datos_abiertos_2026_04.csv
Viajes mayo 2026	https://mibici.net/site/assets/files/125943/datos_abiertos_2026_05.csv
Viajes junio 2026	https://mibici.net/site/assets/files/127734/datos_abiertos_2026_06.csv
Viajes julio 2026	https://mibici.net/site/assets/files/129006/datos_abiertos_2026_07.csv
Viajes agosto 2026	https://mibici.net/site/assets/files/130362/datos_abiertos_2026_08.csv
Viajes septiembre 2026	https://mibici.net/site/assets/files/131825/datos_abiertos_2026_09.csv
Nomenclatura de estaciones	https://mibici.net/site/assets/files/1118/nomenclatura_2026_09.csv

Tabla 2. *Archivos que descarga la ingesta.*

2.3 Tamaño real obtenido

Dato	Valor
Meses	12 (octubre 2025 – septiembre 2026)
Viajes en los CSV originales	4,497,877
Viajes después de la limpieza	4,332,379
Viajes descartados	165,498 (3.7 %)
Estaciones en la nomenclatura	484, con ubicación y capacidad
Estaciones con viajes en el periodo	471 (469 de la nomenclatura y 2 que no aparecen en ella); 15 estaciones de la nomenclatura no tuvieron viajes
Tamaño en disco	390 MB en los 12 CSV de viajes → 31 MB en Parquet; 27 KB el Parquet de estaciones

Tabla 3. *Tamaño de los datos después de correr **make data**.*

La ingesta (**mi_bici/dataset.py**) normaliza las diferencias entre archivos: reconoce cada columna por un patrón de su nombre, porque los encabezados cambian entre años y traen acentos y mayúsculas; lee en UTF-8 y, si falla, en Latin-1, y convierte las fechas aunque vengan en formato ISO o **DD/MM/AAAA**, con o sin segundos, para guardarlas en un formato uniforme (**AAAA-MM-DD HH:MM**). Después aplica las reglas de la Tabla 4. Si una ejecución llegara a descartar más del 5 % de las filas, deja un error en el registro para revisarla.

Regla	Por qué
Quitar duraciones negativas	Son errores de registro
Quitar viajes de menos de 2 minutos que salen y regresan a la misma estación	El usuario sacó la bicicleta y la volvió a anclar: no es un viaje real
Quitar viajes de más de 12 horas	Los términos de MiBici suspenden la cuenta tras 8 horas de uso continuo y no reembolsan los cargos si la bicicleta no se devuelve en 12 horas (MiBici, s.f.-b): no es un traslado normal
Quitar duplicados del identificador del viaje	Un viaje no puede contarse dos veces en la demanda

Regla	Por qué
Quitar años de nacimiento anteriores a 1920	Edades imposibles
Género vacío “NE” (no especificado)	Se conserva el viaje, que sí cuenta para la demanda

Tabla 4. *Reglas de limpieza de los viajes.*

Prueba de reproducibilidad. Corrimos la ingesta en un clon limpio, siguiendo solo el **README**, en otra computadora. Los 12 archivos de viajes salieron idénticos byte por byte: el mismo hash que registra el manifiesto, con 4,332,379 viajes. El Parquet de estaciones conservó las mismas 484 filas, columnas y tamaño, pero su hash cambió. En una misma computadora el hash se repite, así que la diferencia viene del entorno con que se escribe el archivo y no de los datos. Como mejora, calcularemos la versión sobre el contenido de las tablas y no sobre los bytes del archivo. Para la corrida oficial usamos el manifiesto versionado en main, así que todos los runs comparten el mismo data_version.

2.4 Variables originales y cómo construimos cada objetivo

Cada archivo de viajes tiene ocho columnas: identificador del viaje, identificador del usuario, género, año de nacimiento, fecha y hora de inicio y de fin, y estaciones de origen y destino. La nomenclatura trae el identificador, el nombre, la latitud, la longitud y la capacidad de cada estación.

- **Usamos** el inicio, el fin, el origen y el destino de cada viaje (además de la duración, que calculamos para limpiar), y todas las columnas de la nomenclatura: la capacidad en P1 y P2, la ubicación para el mapa de P3 y el prefijo del nombre (GDL, ZPN, TLQ) como municipio para el baseline de P3.
- **Solo para describir** los grupos de P3 usamos el género y el año de nacimiento, agregados por estación.
- **Descartamos** los identificadores del viaje (después de quitar duplicados) y del usuario.

Ninguna pregunta tiene una etiqueta ya hecha en los datos; las construimos a partir de los viajes:

- **P1**. Contamos los viajes que inician en cada estación cada día. Un día sin viajes cuenta como 0, desde la primera aparición de la estación: la demanda cero es información, no un dato faltante.
- **P2**. Contamos las salidas de cada estación según la hora de inicio y las llegadas según la hora de fin, en bloques de 3 horas. El flujo neto es llegadas menos salidas, y la etiqueta riesgo vale 1 si su valor absoluto alcanza el 40 % de la capacidad (sección 1.1).
- **P3**. No hay objetivo. Construimos un perfil de uso por estación con los viajes del periodo de entrenamiento e incluimos solo las estaciones con al menos 30 días con viajes, para que su perfil sea representativo.

2.5 Condiciones de uso (licencia) y tratamiento de datos personales

Condiciones de uso. El portal invita: «¡Explora, experimenta y comparte los datos de uso de Mi Bici!» (MiBici, s.f.-a), pero no publica una licencia explícita ni condiciones de atribución. El enlace «Manual para uso de datos abiertos» descarga el archivo de nomenclatura en lugar de un manual (consultado el 18 de octubre de 2026). El pie del sitio indica que el sistema lo opera BKT Bici Pública y lo administra la Agencia Metropolitana de Servicios de Infraestructura para la Movilidad del Área Metropolitana de Guadalajara. Con eso, usamos los datos solo con fines académicos, citamos la fuente y no los redistribuimos: el repositorio descarga los archivos desde los enlaces oficiales y no guarda ninguno en Git.

Datos personales. El aviso de privacidad del sitio establece que la Agencia trata los datos personales conforme a la ley de protección de datos personales en posesión de sujetos obligados del Estado de Jalisco y a la ley general correspondiente. Ese aviso considera dato personal cualquier información de una persona identificada o **identificable**, aunque la identificación sea indirecta (Agencia Metropolitana de Servicios de Infraestructura para la Movilidad del Área Metropolitana de Guadalajara, s.f.). Los viajes traen un identificador de usuario que se repite en todos sus viajes, junto con su género y año de nacimiento. Combinados con las estaciones y horarios exactos, permitirían reconocer las rutas de una persona. Por eso:

1. La ingesta elimina el identificador del usuario y el del viaje; los datos procesados ya no los tienen.
2. El género y el año de nacimiento no entran a ningún modelo. Solo se agregan por estación (porcentaje de usuarias y edad mediana) para describir los grupos de P3.
3. MLflow no recibe datos de personas. Los parámetros son de configuración, las métricas son agregadas y los artefactos se resumen por estación o grupo. El ejemplo de entrada que se guarda con cada modelo solo tiene variables agregadas.
4. Los datos no se versionan: .gitignore excluye data/ (salvo el manifiesto), y las salidas de los notebooks no muestran viajes individuales.

2.6 Partición sin información del futuro

Separamos los datos por fecha, no al azar. Entrenamiento: octubre de 2025 a julio de 2026 (10 meses). Prueba: agosto y septiembre de 2026 (2 meses). El corte es el 1 de agosto de 2026 (**CORTE_PRUEBA = "2026-08-01"** en **config.py**) y la semilla es 42 (**SEMILLA = 42**). La parte inferior de la Figura 2 muestra la partición.

Los modelos se usan para predecir el futuro, así que evaluarlos con los datos más recientes da una estimación más realista que una muestra aleatoria (Treveil et al., 2020, p. 52). Una partición aleatoria mezclaría días de agosto en el entrenamiento y dejaría ver patrones del periodo que se quiere predecir. Además, apartar la prueba desde el inicio evita elegir el modelo con base en ella, el sesgo que Géron (2019, p. 54) llama *data snooping*.
La fuga de información también se cuida dentro de las variables:

- **P1 y P2**: cada fila es el periodo que se quiere predecir, y todas sus variables vienen de periodos anteriores (desplazamiento de al menos un periodo). La media móvil se calcula después del desplazamiento, así que solo incluye periodos ya observados.

En prueba, los rezagos usan los viajes reales de los días previos, que el operador ya conoce cuando hace el pronóstico. El día de la semana y los días festivos se conocen de antemano.

- **P3**: el perfil de cada estación solo usa viajes anteriores al corte, y el escalado se ajusta solo con esos perfiles.

Limitación. El entrenamiento no incluye agosto ni septiembre, y la prueba solo cubre dos meses. Una validación con varias ventanas temporales daría una idea de la estabilidad de las métricas (Treveil et al., 2020, p. 52); queda para las siguientes entregas.

3. Pipeline de ML

La Figura 2 muestra las etapas de cada pregunta, del dato crudo al modelo evaluado. Las tres comparten la ingesta y la limpieza (sección 2.3) y se separan a partir de la agregación. Las variables viven en **mi_bici/features.py**, como funciones con un docstring que explica qué hace cada una y por qué es pertinente para su pregunta; crear variables útiles a partir de los datos crudos es el centro del feature engineering (Maddula, 2025).

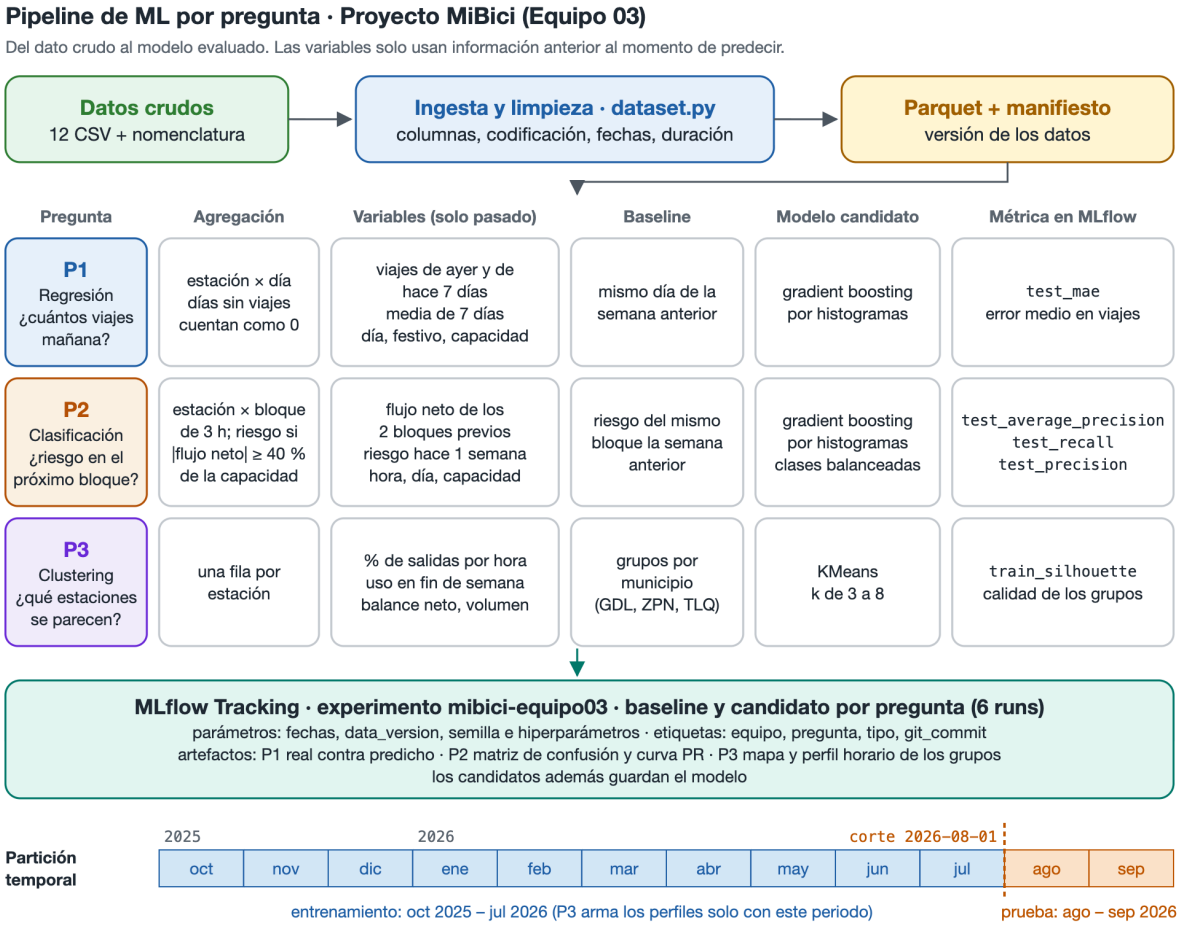


Figura 2. *Pipeline de ML por pregunta y partición temporal de los datos.*

P1 · Demanda diaria (***demanda_diaria***). Agrega los viajes por estación de origen y día, completa con 0 los días sin viajes y descarta los primeros días de cada estación, que todavía no tienen suficiente historia para los rezagos. Después entrena y evalúa con el corte temporal.

Variable	Por qué es pertinente
viajes_lag_1 (viajes de ayer)	La mejor pista del uso de una estación es su uso reciente
viajes_lag_7 (mismo día de la semana anterior)	Captura el patrón semanal: un lunes se parece al lunes anterior. También es el baseline
viajes_media_7 (media de los 7 días previos)	Suaviza días atípicos y refleja el nivel reciente de la estación
dia_semana, festivo	La demanda cambia entre días laborales, fines de semana y días de descanso obligatorio; ambas se conocen de antemano
capacidad	Indica el tamaño de la estación

Tabla 5. *Variables de P1.*

P2 · Riesgo por bloque (***flujo_por_bloque*** y ***etiqueta_riesgo***). Cuenta las salidas y llegadas por estación en bloques de 3 horas, completa con 0 los bloques sin viajes, calcula el flujo neto y la etiqueta, y agrega rezagos.

Variable	Por qué es pertinente
flujo_neto_lag_1, flujo_neto_lag_2	Si una estación ya venía perdiendo o ganando bicicletas, es probable que siga
salidas_lag_1, llegadas_lag_1	Separan un bloque tranquilo de uno con mucho movimiento que se compensa
riesgo_semana_anterior	El mismo bloque de la semana pasada resume el patrón semanal. También es el baseline
hora, dia_semana, festivo	Los picos de entrada y salida dependen de la hora y del tipo de día. El servicio presta bicicletas de 5:00 a 0:59 (MiBici, s.f.-b), así que los bloques de madrugada casi no tienen salidas
capacidad	El mismo flujo pesa más en una estación pequeña

Tabla 6. *Variables de P2.*

P3 · Perfil de cada estación (***perfil_estacion***). Construye una fila por estación con los viajes del periodo de entrenamiento, estandariza las variables y agrupa con KMeans. El escalado es necesario porque KMeans mide distancias, y una variable con valores más grandes dominaría a las demás (scikit-learn developers, s.f.-c). Además, va dentro del mismo *pipeline* del modelo, así que se ajusta solo con los perfiles de entrenamiento.

Variable	Por qué es pertinente
Proporción de salidas por hora del día (24 columnas)	Distingue estaciones de zonas de trabajo, residenciales y de ocio por sus picos de mañana o de tarde
Proporción de salidas en fin de semana	Separa el uso de traslado diario del recreativo
Balance neto relativo, (llegadas − salidas) / total	Separa las estaciones que tienden a vaciarse de las que tienden a llenarse
Viajes por día en escala logarítmica	Mide el volumen sin que domine a las proporciones
Porcentaje de usuarias y edad mediana	Solo describen los grupos; no se usan para formarlos

Tabla 7. *Variables del perfil de P3.*

4. Modelos y métricas

Para cada pregunta definimos un baseline sencillo, sin entrenamiento, y un modelo candidato. Empezar con una regla ingenua da el punto de referencia contra el que se mide cualquier modelo más complejo (Lauchande, 2021, p. 62). Elegimos las métricas con la guía de evaluación de scikit-learn (scikit-learn developers, s.f.-b), porque no hay una métrica única para todos los problemas: hay que escoger la que corresponde a la decisión del negocio (Treveil et al., 2020, p. 51).

Modelo	P1 · Demanda	P2 · Riesgo	P3 · Grupos
Baseline	Los viajes del mismo día de la semana anterior	El riesgo del mismo bloque la semana anterior	Agrupar las estaciones por municipio
Candidato	HistGradientBoostingRegressor	HistGradientBoostingClassifier con clases balanceadas	KMeans, probando k de 3 a 8 y eligiendo el de mayor silueta
Métrica principal	test_mae	test_average_precision	train_silhouette

Modelo	P1 · Demanda	P2 · Riesgo	P3 · Grupos
Métricas complementarias	test_rmse, test_r2	test_recall, test_precision, test_f1 (umbral 0.5)	train_davies_bouldin, train_calinski_harabasz

Tabla 8. *Baseline, candidato y métricas por pregunta.*

Por qué esos baselines. Un baseline simple permite saber si un modelo más complejo realmente aporta (Huyen, 2022). Los tres son las reglas que usaría el operador sin un modelo: suponer que esta semana se repite la anterior (P1 y P2) o dividir la red por municipio (P3). Un modelo que no los supera no aporta nada.

Por qué esos candidatos. Los modelos de gradient boosting por histogramas de scikit-learn son mucho más rápidos que el gradient boosting clásico cuando hay decenas de miles de filas o más (scikit-learn developers, s.f.-d), y P1 tiene 112,084 filas de entrenamiento y P2, 896,067. Combinan variables de distinta escala sin estandarizarlas y capturan interacciones, por ejemplo entre la hora y el tipo de día. En P2 balanceamos las clases (**class_weight="balanced"**) porque solo el 4.8 % de los bloques de entrenamiento está en riesgo. KMeans es el algoritmo de propósito general de la guía de clustering y escala bien con muchas observaciones (scikit-learn developers, s.f.-a); como pide fijar el número de grupos, probamos de 3 a 8. En esta entrega los candidatos usan los hiperparámetros por defecto, que quedan registrados en cada run; ajustarlos queda para la siguiente.

Por qué esas métricas.

- **P1**: **error absoluto medio (MAE).** Se lee en viajes (“el pronóstico se equivoca en promedio por 5 viajes al día”), que es lo que entiende el operador. Es menos sensible a valores extremos que el RMSE (Géron, 2019, p. 44); registramos también el RMSE, que castiga más los errores grandes, y el R^2. No usamos un error porcentual porque hay días con cero viajes.
- **P2**: **precisión promedio (average precision).** Con tan pocos positivos, la exactitud engaña: un modelo que siempre dice “sin riesgo” acertaría más del 95 % de las veces sin servir de nada (Treveil et al., 2020, p. 44). La precisión promedio resume la curva de precisión y recall en todos los umbrales. El recall y la precisión traducen el umbral de 0.5 a la decisión: el recall dice cuántos riesgos detectamos (un riesgo no detectado es un usuario sin bicicleta o sin anclaje) y la precisión, cuántas visitas del camión serían en vano.
- **P3**: **coeficiente de silueta.** Sin etiquetas reales, solo podemos medir qué tan compactos y separados quedan los grupos. La silueta va de −1 a 1: cerca de 1, los grupos están bien separados, y cerca de 0, se traslapan (scikit-learn developers, s.f.-a). Davies-Bouldin (menor es mejor) y Calinski-Harabasz (mayor es mejor) la complementan. Baseline y candidato se miden en el mismo espacio estandarizado, para compararlos con la misma vara.

Los nombres de las métricas indican el conjunto en el que se calculan: test_ para la prueba de P1 y P2, y train_ para P3, que solo usa el periodo de entrenamiento. En la siguiente etapa revisaremos además el error por municipio, para comprobar que el modelo funciona igual en toda la red (Treveil et al., 2020, p. 53).

5. Arquitectura y herramientas

La Figura 3 muestra los componentes de la solución y cómo se conectan. Los datos van del portal a la ingesta, que guarda los CSV originales sin modificar en **data/raw/** y los datos limpios en Parquet en **data/processed/**. El manifiesto es lo único de **data/** que se versiona en Git. Las variables leen el Parquet, el entrenamiento ajusta y evalúa los modelos, y MLflow registra cada run con la versión de los datos (**data_version**) y el commit de Git (**git_commit**). La configuración, el entorno, los comandos y el control de versiones atraviesan todos los componentes. El Model Registry, el servicio de predicción y el monitoreo quedan para las siguientes entregas.

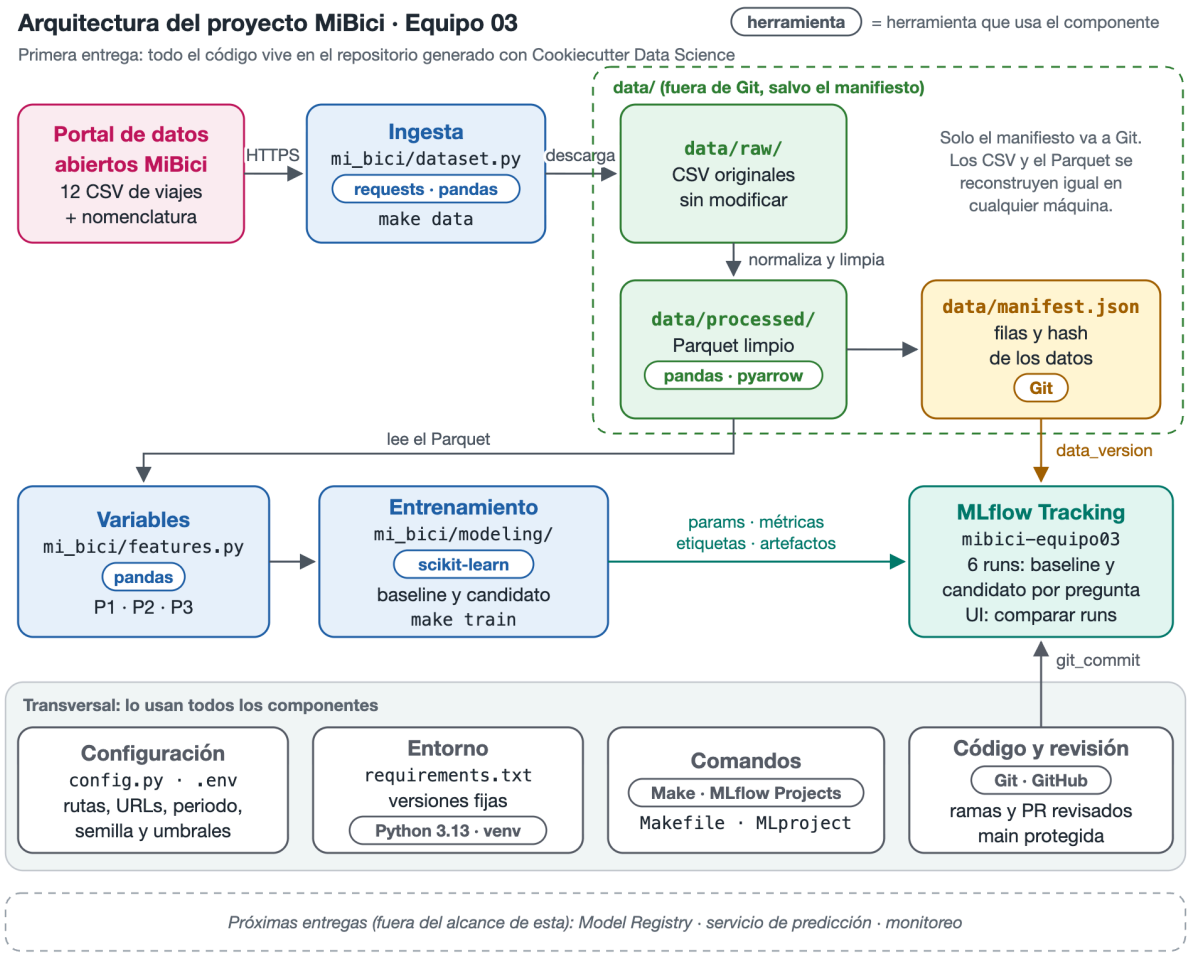


Figura 3. *Arquitectura del proyecto MiBici del Equipo 03.*

Componente y herramienta	Para qué la usamos	Por qué (lecturas del curso)
Estructura: Cookiecutter Data Science 2.3.0 (ccds)	Separar datos, notebooks, código del paquete mi_bici, modelos, reportes y referencias	Una estructura estándar deja que cualquiera encuentre cada cosa y reproduzca el análisis (DrivenData, s.f.-d). Los datos crudos son inmutables y los notebooks son para explorar, no para la lógica final (DrivenData, s.f.-b). Así evitamos el código pegamento y las «selvas de pipelines», dos fuentes de deuda técnica (Sculley et al., 2015, p. 2507)
Entorno: Python 3.13, venv y requirements.txt con versiones fijas	Reconstruir el mismo entorno en cualquier computadora	La reproducibilidad exige controlar las versiones de todo, incluido el entorno de software (Treveil et al., 2020, p. 26). CCDS recomienda construir el proyecto desde un entorno declarado (DrivenData, s.f.-b)
Configuración: config.py y .env	Rutas, URLs, periodo, corte, semilla, umbrales y experimento en un solo lugar; la dirección de MLflow, en una variable de entorno	La configuración dispersa es deuda técnica (Sculley et al., 2015, p. 2508). Los secretos y la configuración local van en .env, fuera del control de versiones (DrivenData, s.f.-b)
Ingesta: requests y pandas en dataset.py	Descargar, normalizar y limpiar sin pasos manuales	Hay que conocer los datos y su procedencia antes de modelar (Wilson, 2022, pp. 401–408); por eso la limpieza vive en la ingesta y no en el código de los modelos
Almacenamiento: Parquet con pyarrow	Guardar 4.3 millones de viajes limpios en 31 MB	Parquet es un formato binario y columnar que pandas lee y escribe directamente, pensado para guardar y leer tablas de forma eficiente (The pandas development team, s.f.)
Versión de los datos: data/manifest.json con hashlib	Registrar filas, tamaño, columnas y hash de los datos; el hash del manifiesto es data_version	Cada versión de un modelo debe poder reproducirse, con sus datos (Treveil et al., 2020, pp. 56–58). Los datos no van en Git, solo su

Componente y herramienta	Para qué la usamos	Por qué (lecturas del curso)
		descripción (DrivenData, s.f.-b). El hash se calcula con hashlib (Python Software Foundation, s.f.)
Variables: pandas en features.py	Funciones documentadas por pregunta, sin información del futuro	Las transformaciones se ajustan solo con el entrenamiento y se aplican igual a la prueba (scikit-learn developers, s.f.-c). El código modular se lee, se prueba y se reutiliza mejor que un script monolítico (Wilson, 2022, pp. 248–263)
Modelado: scikit-learn	Baselines, candidatos (HistGradientBoosting, KMeans), escalado y métricas	Guías de aprendizaje supervisado, clustering y evaluación del enunciado (scikit-learn developers, s.f.-a, s.f.-b, s.f.-d). Una clase base, ExperimentoBase, registra igual las tres preguntas y cada una solo define sus datos, su modelo y su evaluación
Registro de experimentos: MLflow Tracking, con SQLite local	Parámetros, métricas, etiquetas, artefactos y modelos de cada run; comparar runs en la interfaz	Un registro de experimentos guarda los datos, variables y parámetros de cada resultado y permite compararlos lado a lado (Treveil et al., 2020, p. 26). MLflow Tracking ofrece la API y la interfaz para hacerlo (Lauchande, 2021, pp. 9, 21–22; MLflow, s.f.), con un experimento con nombre y runs con nombre descriptivo (Lauchande, 2021, p. 69)
Comandos: Make (make data, make train) y MLproject	Un comando por etapa; MLproject expone la ingesta como entry point load	CCDS usa Make como ejecutor de tareas para correr cada paso con un solo comando (DrivenData, s.f.-c)
Gráficas: matplotlib	Artefactos de cada run (real contra predicho, matriz de confusión, curva PR, mapa y perfil horario)	Los artefactos se registran con log_artifact junto al run (MLflow, s.f.)

Componente y herramienta	Para qué la usamos	Por qué (lecturas del curso)
Pruebas: pytest	Probar la limpieza y la lectura de fechas	Las pruebas capturan el comportamiento del código para poder cambiarlo sin romperlo (Wilson, 2022, pp. 264–268)
Colaboración: Git y GitHub	Ramas, Pull Requests revisados y main protegida	GitHub Flow, ramas protegidas y revisiones de Pull Requests (GitHub, s.f.-a, s.f.-b, s.f.-c, s.f.-d); commits con Conventional Commits (Conventional Commits, s.f.)

Tabla 9. *Herramientas de cada componente y su justificación.*

5.1 Repositorio y flujo de trabajo

Generamos el repositorio con **ccds** (DrivenData, s.f.-a), con **include_code_scaffold** en Yes, y respetamos su separación: datos en **data/**, notebooks en **notebooks/**, código en **mi_bici/**, modelos en **models/** y figuras en **reports/figures/**. El ML Canvas y los diagramas están en **references/. .gitignore** excluye los datos de MiBici, **mlflow.db**, **mlruns/**, **mlartifacts/**, **.env** y los modelos serializados.

Seguimos GitHub Flow (GitHub, s.f.-d): **main** está protegida (GitHub, s.f.-b), cada cambio sale en su propia rama (**tipo/descripcion**, por ejemplo **feat/variables-y-runs**) y entra solo por Pull Request con la revisión de otro integrante (GitHub, s.f.-c). Comprobamos que GitHub rechaza un push directo a **main**. Los títulos de todos los Pull Requests siguen Conventional Commits en español; los primeros commits de la ingesta no siguieron la convención y no los reescribimos. Los Pull Requests de cada entregable ligan su issue con Closes #N o Refs #N. El **README** documenta la instalación, los comandos y este flujo. Los cuatro integrantes tienen commits en **main**, abrieron al menos un Pull Request y revisaron al menos uno de otro integrante (Tabla 10). Este documento entra al repositorio, en **reports/**, con su propio Pull Request.

Integrante	Abrió	Revisó
Miguel Ángel Ramírez Castillo	#17	#1, #23, #24
Gerardo González Martínez	#1, #19, #23 (cerrado, lo reemplazó el #25), #24, #25, #27, #30	#17, #21, #26, #29
Anaí Lael Barrera Magaña	#21, #29	#18, #20, #28

Integrante	Abrió	Revisó
Sebastián Estrada García	#18, #20, #26, #28	#19, #25, #27, #30

Tabla 10. *Pull Requests abiertos y revisados por integrante.*

6. Resultados del experimento base

Corrimos **make train** en un clon limpio de **main**, siguiendo solo el **README**. El comando registró seis runs en el experimento **mibici-equipo03**, un baseline y un candidato por pregunta, todos con el commit **0d6e728**, sin cambios sin guardar (**git_dirty = False**) y con la misma versión de los datos (**data_version = 73468430172e**). Cada run guarda como parámetros el rango de fechas, el corte de prueba, **data_version**, la semilla, el número de filas de entrenamiento, el modelo y sus hiperparámetros; como etiquetas, el equipo, la pregunta, el tipo de modelo, el commit de Git y si había cambios sin guardar. Las métricas coinciden, a tres decimales, con las de una corrida anterior en la computadora de otro integrante, con el mismo **data_version**.

Pregunta	Run	Filas de entrenamiento	Métrica principal	Complementarias
P1	P1-baseline-semana-anterior	112,084	test_mae = 6.907	test_rmse = 12.993; test_r2 = 0.858
P1	P1-hgb-rezagos	112,084	test_mae = 5.563	test_rmse = 10.402; test_r2 = 0.909
P2	P2-baseline-semana-anterior	896,067	test_average_precision = 0.168	test_recall = 0.365; test_precision = 0.385; test_f1 = 0.375
P2	P2-hgb-balanced	896,067	test_average_precision = 0.376	test_recall = 0.870; test_precision = 0.146; test_f1 = 0.250
P3	P3-baseline-municipio	426 estaciones	train_silhouette = 0.017	train_davies_bouldin = 5.141;

Pregunta	Run	Filas de entrenamiento	Métrica principal	Complementarias
				train_calinski_harabasz = 5.0
P3	P3-kmeans-perfil-horario	426 estaciones	train_silhouette = 0.096 (k = 3)	train_davies_bouldin = 2.552; train_calinski_harabasz = 39.9

Tabla 11. *Resultados del experimento base (prueba: agosto y septiembre de 2026).*

En P2, el 4.8 % de los bloques de entrenamiento y el 4.4 % de los de prueba están en riesgo (**train_tasa_riesgo** y **test_tasa_riesgo**).

P1. El candidato reduce el error medio de 6.9 a 5.6 viajes por estación y día (19.5 % menos que el baseline) y el RMSE de 13.0 a 10.4; además explica el 91 % de la variación de la demanda (R^2 de 0.909 contra 0.858). Podemos observar que los viajes de ayer y la media de la última semana aportan información que el mismo día de la semana anterior no tiene. En la gráfica de viajes reales contra predichos que guarda el run, sumada para todo el sistema, el modelo sigue bien el patrón semanal, pero se queda corto en los días de mayor demanda de la segunda mitad de septiembre, cuando el sistema pasa de unos 14 mil a cerca de 20 mil viajes diarios: los rezagos solo ven el nivel de los días anteriores.

P2. El candidato duplica la precisión promedio del baseline (0.376 contra 0.168). Como referencia, un clasificador al azar obtendría una precisión promedio cercana a la proporción de bloques en riesgo, 0.044 en la prueba. Con el umbral de 0.5 y clases balanceadas, el candidato detecta 87 de cada 100 bloques en riesgo (recall de 0.870, contra 0.365 del baseline), pero solo 15 de cada 100 alertas son reales (precisión de 0.146, contra 0.385). Por eso su F1 (0.250) es menor que el del baseline (0.375). El modelo prefiere detectar más riesgos a cambio de más falsas alarmas, lo que conviene si una visita innecesaria del camión cuesta menos que un usuario sin bicicleta o sin anclaje. Como la precisión promedio resume todos los umbrales, el operador puede subir el umbral para reducir las alarmas según la capacidad del camión; elegirlo con un conjunto de validación queda para la siguiente entrega.

P3. Los grupos por patrón de uso son más coherentes que los municipios en las tres métricas (silueta de 0.096 contra 0.017, Davies-Bouldin de 2.55 contra 5.14 y Calinski-Harabasz de 39.9 contra 5.0), pero la silueta sigue cerca de 0: los grupos se traslapan. El mejor k fue 3, el mínimo del rango, y ningún otro k superó esa silueta. El perfil horario (Figura 4) muestra tres patrones:

- **Grupo 0 (179 estaciones)**: pico de salidas a las 8 h. Son estaciones de donde la gente sale por la mañana, lo que sugiere zonas residenciales.

- **Grupo 2 (186 estaciones):** pico a las 18 h y uso alto por la tarde, el patrón inverso, que sugiere zonas de trabajo o estudio. Es el grupo con más viajes por día.
- **Grupo 1 (61 estaciones):** salidas repartidas entre las 7 y las 13 h y mucho menos volumen. En el mapa (Figura 7) quedan sobre todo en la periferia.

El porcentaje de usuarias (24 % a 28 %) y la edad mediana (32 años) casi no cambian entre grupos. Interpretamos estos resultados como patrones de uso que cambian de forma continua más que como tipos bien separados: los grupos sirven para describir la red, no como una clasificación definitiva.

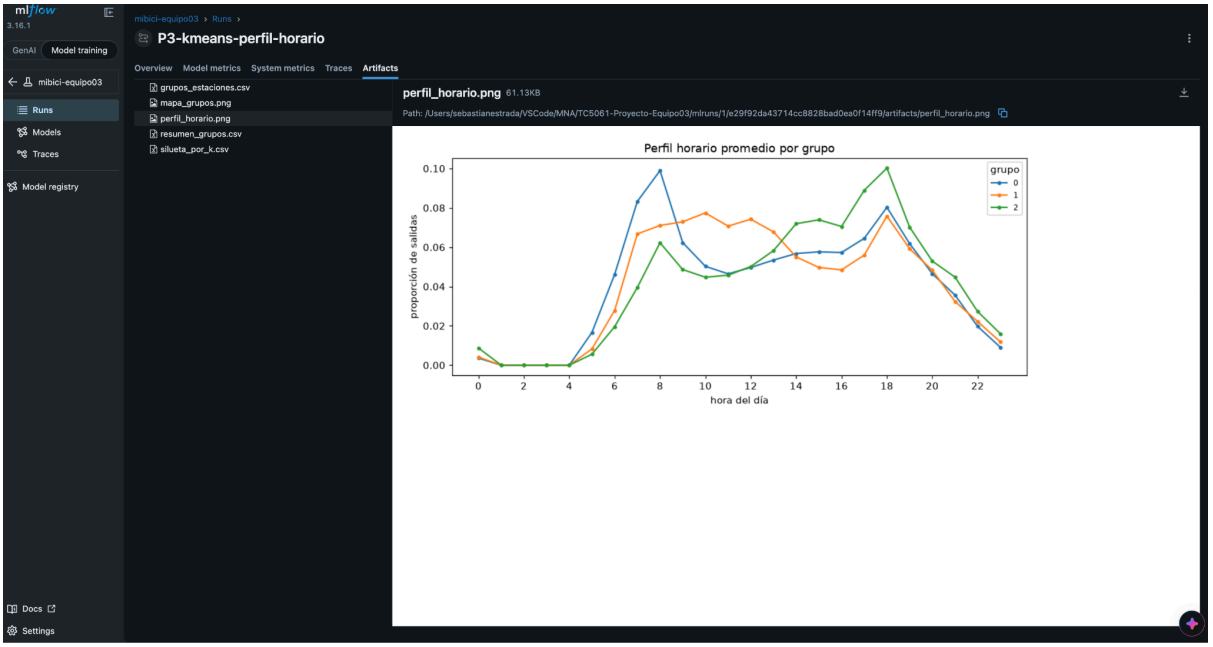


Figura 4. *Perfil horario promedio de salidas por grupo de estaciones (artefacto del run P3-kmeans-perfil-horario).*

Limitaciones y siguientes pasos.

- La etiqueta de P2 es una aproximación sin inventario (sección 1.1). Separar vaciarse de llenarse y ajustar el umbral con el operador son las primeras mejoras.
- Los datos no incluyen clima, eventos ni movimientos de rebalanceo, que también explican la demanda.
- La prueba cubre solo dos meses. Validar con varias ventanas temporales y ajustar hiperparámetros, sin tocar la prueba, queda para la siguiente entrega.
- En P3 probaremos k = 2, otras variables del perfil y otros algoritmos de la guía de clustering (scikit-learn developers, s.f.-a).
- Las siguientes entregas agregan el Model Registry, un servicio de predicción y el monitoreo.

7. Evidencia: MLflow y repositorio

Las capturas corresponden a la corrida oficial de make train desde main, cuyo commit aparece en la etiqueta git_commit de cada run.

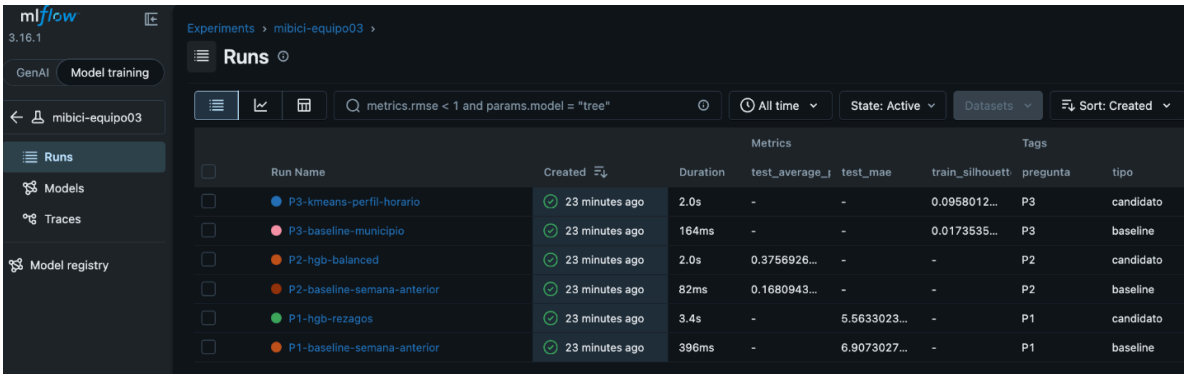


Figura 5. *Lista de runs del experimento **mibici-equipo03** en MLflow.*

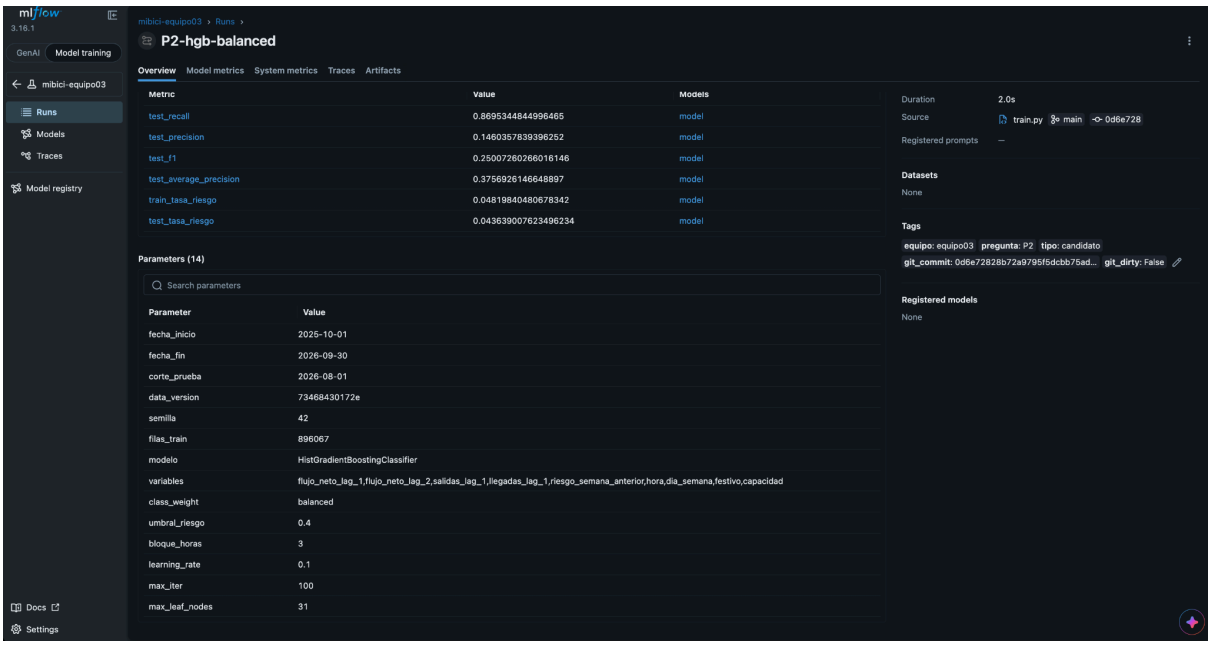


Figura 6. *Detalle del run **P2-hgb-balanced**: parámetros, métricas y etiquetas.*

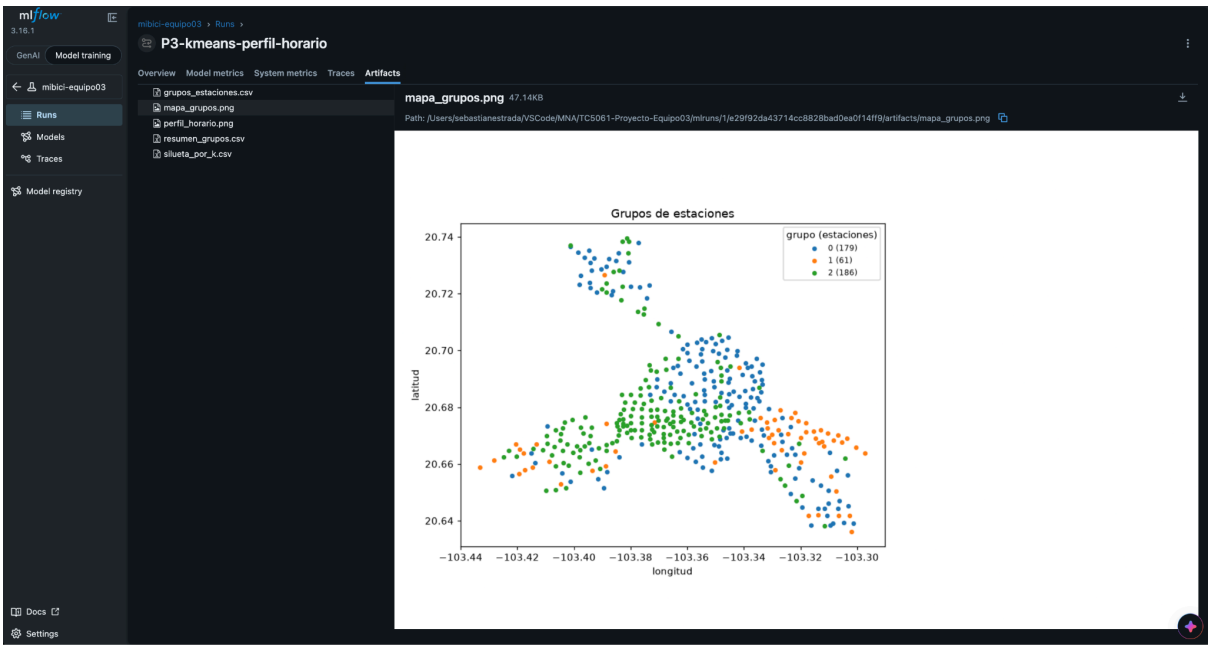


Figura 7. *Artefacto del run P3-kmeans-perfil-horario en MLflow: grupos de estaciones en el mapa.*

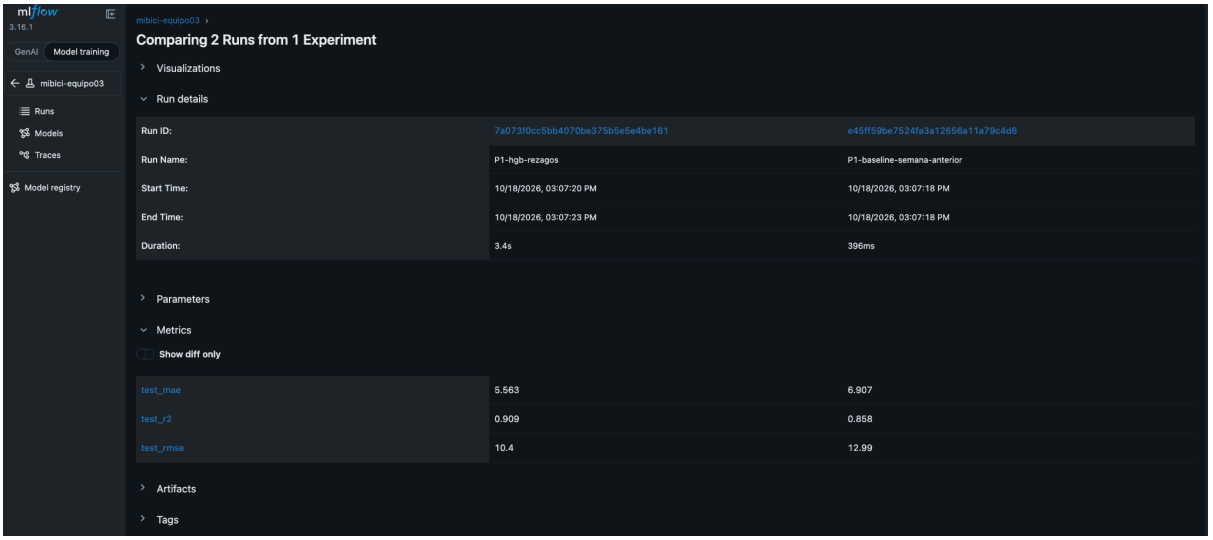


Figura 8. *Comparación del baseline y el candidato de P1 en MLflow: métricas de prueba lado a lado.*

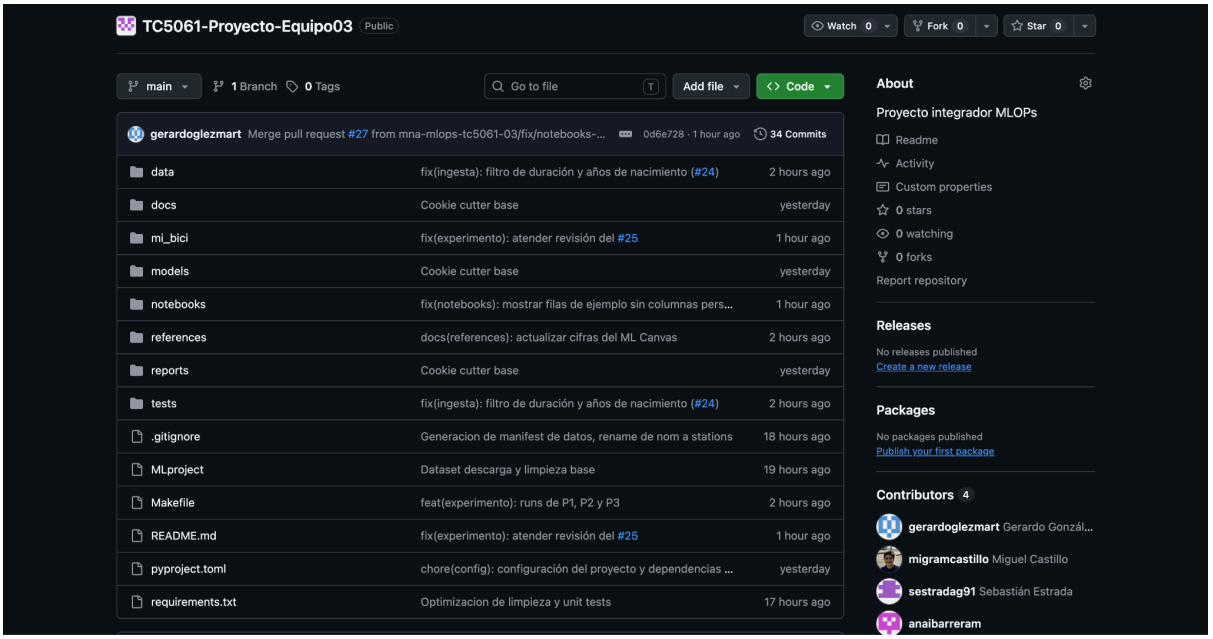


Figura 9. *Estructura del repositorio TC5061-Proyecto-Equipo03 generada con Cookiecutter Data Science.*

Referencias

Agencia Metropolitana de Servicios de Infraestructura para la Movilidad del Área Metropolitana de Guadalajara. (s.f.). *Aviso de privacidad integral*. MiBici. Recuperado el 18 de octubre de 2026, de https://mibici.net/es/aviso-de-privacidad/

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje grande], utilizado mediante Claude Code para apoyo en la redacción, la generación de diagramas y la revisión de código. https://claude.com/claude-code

Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. Recuperado el 17 de octubre de 2026, de https://www.conventionalcommits.org/es/v1.0.0/

Dorard, L. (2015). *The Machine Learning Canvas* (versión 1.2) [Plantilla]. OWNML. https://www.ownml.co/machine-learning-canvas

Dorard, L. (2018, 9 de marzo). *From data to AI with the Machine Learning Canvas (Part I)*. Own Machine Learning. https://medium.com/louis-dorard/from-data-to-ai-with-the-machine-learning-canvas-part-i-d171b867b047

DrivenData. (s.f.-a). *Cookiecutter Data Science*. Recuperado el 10 de octubre de 2026, de https://cookiecutter-data-science.drivendata.org/

DrivenData. (s.f.-b). *Opinions*. Cookiecutter Data Science. Recuperado el 10 de octubre de 2026, de https://cookiecutter-data-science.drivendata.org/opinions/

DrivenData. (s.f.-c). *Using the template*. Cookiecutter Data Science. Recuperado el 10 de octubre de 2026, de https://cookiecutter-data-science.drivendata.org/using-the-template/

DrivenData. (s.f.-d). *Why ccds?* Cookiecutter Data Science. Recuperado el 10 de octubre de 2026, de https://cookiecutter-data-science.drivendata.org/why/

Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems* (2a ed.). O'Reilly Media.

GitHub. (s.f.-a). *About Git*. GitHub Docs. Recuperado el 17 de octubre de 2026, de https://docs.github.com/en/get-started/using-git/about-git

GitHub. (s.f.-b). *About protected branches*. GitHub Docs. Recuperado el 17 de octubre de 2026, de https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

GitHub. (s.f.-c). *About pull request reviews*. GitHub Docs. Recuperado el 17 de octubre de 2026, de https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

GitHub. (s.f.-d). *GitHub flow*. GitHub Docs. Recuperado el 17 de octubre de 2026, de https://docs.github.com/en/get-started/using-github/github-flow

Huyen, C. (2022). *Designing machine learning systems: An iterative process for production-ready applications*. O'Reilly Media.

Lauchande, N. (2021). *Machine learning engineering with MLflow: Manage the end-to-end machine learning life cycle with MLflow*. Packt Publishing.

Maddula, S. (2025, 19 de marzo). *Feature engineering in machine learning: A practical guide*. DataCamp. https://www.datacamp.com/tutorial/feature-engineering

MiBici. (s.f.-a). *Datos abiertos* [Conjunto de datos]. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. Recuperado el 17 de octubre de 2026, de https://mibici.net/es/datos-abiertos/

MiBici. (s.f.-b). *Términos y condiciones*. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. Recuperado el 18 de octubre de 2026, de https://mibici.net/es/termino-y-condiciones/

MLflow. (s.f.). *MLflow Tracking* [Documentación]. Recuperado el 17 de octubre de 2026, de https://mlflow.org/docs/latest/ml/tracking/

The pandas development team. (s.f.). *IO tools (text, CSV, HDF5, …): Parquet*. pandas documentation. Recuperado el 17 de octubre de 2026, de https://pandas.pydata.org/docs/user_guide/io.html#parquet

Python Software Foundation. (s.f.). *hashlib — Secure hashes and message digests*. Python 3 documentation. Recuperado el 17 de octubre de 2026, de https://docs.python.org/3/library/hashlib.html

scikit-learn developers. (s.f.-a). *Clustering*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/modules/clustering.html

scikit-learn developers. (s.f.-b). *Metrics and scoring: Quantifying the quality of predictions*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/modules/model_evaluation.html

scikit-learn developers. (s.f.-c). *Preprocessing data*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/modules/preprocessing.html

scikit-learn developers. (s.f.-d). *Supervised learning*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/supervised_learning.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. En C. Cortes, N. Lawrence, D. Lee, M. Sugiyama y R. Garnett (Eds.), *Advances in Neural Information Processing Systems 28* (pp. 2503–2511). Curran Associates.

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.

Wilson, B. (2022). *Machine learning engineering in action*. Manning Publications.